

TRƯỜNG ĐẠI HỌC NAM CẦN THƠ
KHOA CÔNG NGHỆ THÔNG TIN

MODULE 2
MẢNG - ARRAY



Bùi Thị Diễm Trinh
Khoa CNTT, Đại Học Nam Cần Thơ
Email: btdtrinh@nctu.edu.vn



NỘI DUNG CHÍNH

1. Mảng một chiều



2. Mảng hai chiều



3. Mảng với kích thước động



4. Mảng các đối tượng



5. Sắp xếp các phần tử của mảng



PHẦN 1. MẢNG 1 CHIỀU





KHÁI NIỆM

- Là cấu trúc cho phép lưu trữ nhiều phần tử có cùng kiểu dữ liệu.



- Để truy xuất giá trị của phần tử thì cần biết chỉ số của phần tử đó. Chỉ số được đánh số từ 0.
 - Các thao tác trên mảng
 - ✓ Khai báo
 - ✓ Truy xuất (đọc/ghi) phần tử
 - ✓ Lấy số phần tử
 - ✓ Sắp xếp các phần tử mảng



KHAI BÁO

- Khai báo: <kiểu dữ liệu> <tên mảng> [];
- Tạo mảng: <tên mảng> [] = new [kiểu dữ liệu] [<số phần tử>];

```
int mangSoNguyen [];
```

```
double [] mang10SoThuc = new double [10];
```

- Khai báo và khởi tạo giá trị:

```
<kiểu dữ liệu>[] <tên mảng> = new [<kiểu dữ liệu>[]]{danh  
sách phần tử}
```

```
double [] d1 = new double []{2, 3, 4, 5, 6};
```

```
double [] d2 = {2, 3, 4, 5, 6};
```



LẤY SỐ PHẦN TỬ

- Sử dụng thuộc tính `length` để lấy số phần tử của mảng
- Ví dụ: Chương trình khởi tạo mảng với tập phần tử cố định, thực hiện duyệt mảng sử dụng thuộc tính `length`.

```
public class viDuMang_Length {  
  
    public static void main(String args[]) {  
        int[] mangSoNguyen = {1, 6, 9, -10};  
        System.out.print("Mang duoc khoi tao la: ");  
        for(int i=0; i< mangSoNguyen.length; i++)  
            System.out.print(mangSoNguyen[i]+" ");  
    }  
}
```

```
C:\GiangDay\ViDu_Java>java viDuMang_Length  
Mang duoc khoi tao la: 1 6 9 -10
```




TRUY XUẤT PHẦN TỬ

- Các phần tử của mảng được truy xuất thông qua chỉ số.
- Ví dụ: Chương trình nhập vào kích thước mảng, các phần tử kiểu số nguyên và in ra màn hình mảng vừa nhập.

```
import java.util.Scanner;  
public class viDuNhapXuatMang {
```

```
    public static void main(String args[]) {  
        Scanner input = new Scanner(System.in);  
        System.out.println("Nhap vao kích thước của mảng: ");  
        int n = input.nextInt();  
        int [] mangSoNguyen = new int[n];  
        for(int i=0; i<n; i++){  
            System.out.println("Nhap vao phần tử thu "+(i+1)+" mảng: ");  
            mangSoNguyen[i] = input.nextInt();  
        }  
        System.out.print("Mảng nhập vào là: ");  
        for(int i=0; i<n; i++){  
            System.out.print(mangSoNguyen[i]+" ");  
        }  
    }  
}
```

khai báo mảng với kích
thước được nhập từ
bàn phím

nhập giá trị cho từng
phần tử mảng

in giá trị phần tử ra
màn hình

```
C:\GiangDay\ViDu_Java>java viDuNhapXuatMang  
Nhap vao kích thước của mảng:  
4  
Nhap vao phần tử thu 1 mảng:  
9  
Nhap vao phần tử thu 2 mảng:  
-8  
Nhap vao phần tử thu 3 mảng:  
7  
Nhap vao phần tử thu 4 mảng:  
3  
Mảng nhập vào là: 9 -8 7 3
```

VÍ DỤ MẢNG NGẪU NHIÊN

```
package vidu.chuong3.mangmotchieu;
import java.util.Random;
import java.util.Scanner;
public class viDuMangNgauNhiem {
public static void main(String[] args) {
    int n; //Kích thước mảng 1 chiều
    int []a; //Mảng số nguyên 1 chiều có tên a
    Scanner sc = new Scanner(System.in);
    Random rd = new Random();
    do {
        System.out.print("Nhập số lượng phần tử mảng: n = ");
        n = sc.nextInt();
    }while(n<=0 || n>=100);
    a = new int[n];
    for (int i = 0; i < n; i++) {
        a[i] = rd.nextInt(99)-1;
        System.out.print(a[i]+"\\t");
    }
}
```




BÀI TẬP

- Bài 1. Tìm giá trị nhỏ nhất, lớn nhất của mảng số nguyên: Yêu cầu nhập vào mảng một chiều , tìm giá trị lớn nhất, nhỏ nhất
- Bài 2. Viết hàm tìm giá trị dương đầu tiên trong 1 chiều các số thực
- Bài 3. Viết hàm tìm số chẵn cuối cùng trong mảng 1 chiều số nguyên
- Bài 4. Viết hàm tìm vị trí của giá trị nhỏ nhất đầu tiên tìm thấy trong mảng 1 chiều
- Bài 5. Viết hàm Tìm vị trí của phần tử chẵn đầu tiên tìm thấy trong mảng số nguyên
- Bài 6. Viết hàm Tìm giá trị dương nhỏ nhất trong mảng số nguyên.
- Bài 7. Viết hàm nhập vào mảng 1 chiều số nguyên. Liệt kê các vị trí mà giá trị tại đó là số chính phương trong mảng 1 chiều số nguyên
- Bài 8. Viết hàm nhập vào mảng 1 chiều số nguyên. Liệt kê các vị trí mà giá trị tại đó là số nguyên tố trong mảng 1 chiều số nguyên đó, cho biết tổng các số nguyên tố tìm thấy đó

PHẦN 2. MẢNG 2 CHIỀU





KHÁI NIỆM

- Còn được gọi là **ma trận**, là một mảng các mảng 1 chiều. Mảng 2 chiều giống như 1 cái bảng, mảng 1 chiều là các dòng trong bảng và các phần tử của mảng là các ô.
- Phần tử của mảng 2 chiều được truy xuất thông qua 2 chỉ số: **chỉ số dòng** và **chỉ số cột**

	Cột 0	Cột 1	Cột 2	Cột 3
Dòng 0	M[0][0] 7	M[0][1] 2	M[0][2] 9	M[0][3] 0
Dòng 1	M[1][0] 9	M[1][1] 5	M[1][2] 4	M[1][3] 1
Dòng 2	M[2][0] 8	M[2][1] 0	M[2][2] 3	M[2][3] 6

Tên mảng —————

Chỉ số cột —————

Chỉ số dòng —————



KHAI BÁO

- Tạo mảng với số cột ở mỗi dòng là giống nhau

`<kiểu dữ liệu> [][] <tên mảng> = new <kiểu dữ liệu> [
<số phần tử>] [<số phần tử>];`

○ Ví dụ: `int [][] mang2Chieu = new int[2][3];`

- Tạo mảng với số cột ở mỗi dòng là khác nhau.

`<kiểu dữ liệu> [][] <tên mảng> = new <kiểu dữ liệu> [
<số phần tử>] [];`

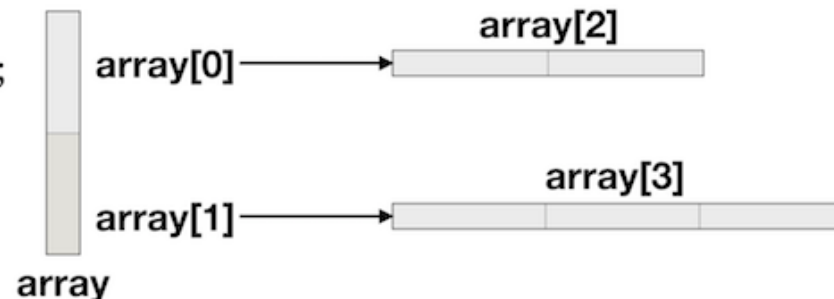
`<tên mảng> [0] = new <kiểu dữ liệu> [<số phần tử>];`

■ ...

```
int[][] array = new int[2][];
```

```
array[0] = new int[2];
```

```
array[1] = new int[3];
```





VÍ DỤ

- Tạo mảng 2 chiều theo quy tắc: số phần tử của mỗi dòng là số thứ tự của dòng đó, các phần tử ở mỗi dòng đều có giá trị là số thứ tự của dòng đó.

```
import java.util.Scanner;  
public class viDuMang2Chieu {
```

```
    public static void main(String args[]) {  
        Scanner input = new Scanner(System.in);  
        System.out.println("Nhap vao so dong cua mang 2 chieu: ");  
        int n = input.nextInt();  
        int [][] mang2Chieu = new int[n][];  
        for(int i = 0; i < n; i++){  
            mang2Chieu[i] = new int[i+1];  
            for(int j=0; j< i+1; j++){  
                mang2Chieu[i][j] = i+1;  
            }  
        }  
        System.out.println("Mang 2 chieu duoc khoi tao la: ");  
        for(int i=0; i< mang2Chieu.length; i++){  
            for(int j = 0; j< mang2Chieu[i].length; j++){  
                System.out.print(mang2Chieu[i][j]+ " ");  
            }  
            System.out.println();  
        }  
    }
```

khởi tạo mảng 2 chiều với
số dòng nhập từ bàn phím

mỗi dòng khởi tạo số
cột là STT của dòng

gán giá trị cho mỗi
phần tử

in giá trị các phần tử
mỗi dòng

xuống dòng để in dòng
mới

```
C:\GiangDay\ViDu_Java>java viDuMang2Chieu  
Nhap vao so dong cua mang 2 chieu:  
4  
Mang 2 chieu duoc khoi tao la:  
1  
2 2  
3 3 3  
4 4 4 4
```

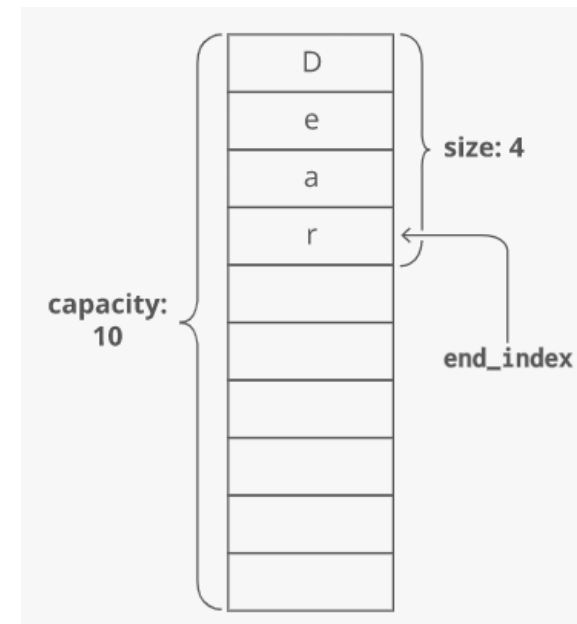

PHẦN 3. MẢNG VỚI KÍCH THƯỚC ĐỘNG





KHÁI NIỆM

- Là mảng trong đó **kích thước của mảng không xác định trước và thay đổi được**.
- Với mảng kích thước động cần phân biệt: **kích thước mảng (size)** và **khả năng lưu trữ (capacity)** của mảng.
- Java hỗ trợ một số lớp tương ứng với kiến trúc mảng kích thước động:
 - ArrayList
 - LinkedList
 - Stack
 - Vector



ARRAYLIST – KHÁI NIỆM

- Là một lớp kế thừa lớp AbstractList và triển khai của List Interface trong Collections Framework.
- Được sử dụng như một mảng động để lưu trữ các phần tử.
- Một số đặc điểm:
 - Có thể chứa các phần tử trùng lặp.
 - Duy trì thứ tự của phần tử được thêm vào.
 - Cho phép truy cập ngẫu nhiên vì lưu dữ liệu theo chỉ mục.
 - Chậm vì cần nhiều sự dịch chuyển nếu bất kỳ phần tử nào bị xóa khỏi danh sách.

ARRAYLIST – CÁCH DÙNG

■ Khởi tạo

- `ArrayList <String> list = new ArrayList <String>();` // tạo một danh sách các phần tử kiểu chuỗi

Phương thức khởi tạo	Mô tả
<code>ArrayList()</code>	Khởi tạo một danh sách mảng trống.
<code>ArrayList(Collection c)</code>	Xây dựng một danh sách mảng được khởi tạo với các phần tử của collection c.
<code>ArrayList(int capacity)</code>	Xây dựng một danh sách mảng mà có dung lượng ban đầu được chỉ định.

ARRAYLIST – CÁCH DÙNG

■ Một số phương thức thường dùng

○ <https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html>

Phương thức	Mô tả
boolean add (Object o)	Thêm phần tử được chỉ định vào cuối một danh sách.
void add (int index, Object element)	Chèn phần tử element tại vị trí index vào danh sách.
Object get (int index)	Trả về phần tử tại vị trí index.
int indexOf (Object o)	Trả về vị trí đầu tiên của phần tử được chỉ định, hoặc -1 nếu danh sách không chứa phần tử này.
int lastIndexOf (Object o)	Trả về vị trí cuối cùng của phần tử được chỉ định, hoặc -1 nếu danh sách không chứa phần tử này.
Object[] toArray ()	Trả về một mảng chứa tất cả các phần tử trong danh sách theo đúng thứ tự.
Object clone ()	Trả về một bản sao của ArrayList.
void clear ()	Xóa tất cả các phần tử từ danh sách.
boolean contains (Object element)	Kết quả trả về là true nếu tìm thấy element trong danh sách, ngược lại trả về false.



ARRAYLIST – VÍ DỤ

- Khởi tạo và duyệt các phần tử của danh sách

```
import java.util.ArrayList;
public class viDuArrayList_1 {

    public static void main(String args[]) {
        ArrayList<String> list = new ArrayList<String>();
        // thêm các phần tử vào list
        list.add("Java");
        list.add("C++");
        list.add("PHP");
        list.add("Java");
        // sử dụng vòng lặp for - hiển thị các phần tử của list
        System.out.println("Cac phan tu trong list la: ");
        for (int i = 0; i < list.size(); i++) {
            System.out.println(list.get(i));
        }
    }
}
```

```
C:\GiangDay\ViDu_Java>java viDuArrayList_1
Cac phan tu trong list la:
Java
C++
PHP
Java
```



ARRAYLIST – VÍ DỤ

■ Cập nhật giá trị của phần tử

```
import java.util.ArrayList;
public class viDuArrayList_2 {

    public static void main(String args[]) {
        ArrayList<String> list = new ArrayList<String>();
        // thêm các phần tử vào list
        list.add("Java");
        list.add("C++");
        list.add("PHP");
        list.add("Pascal");

        System.out.println("list truoC khi cap nhat: ");
        System.out.println(list);
        // cập nhật giá trị cho phần tử có chỉ số là 3 (Pascal)
        list.set(3, "Python");
        System.out.println("list sau khi cap nhat: ");
        System.out.println(list);
    }
}
```

```
C:\GiangDay\ViDu_Java>java viDuArrayList_2
list truoC khi cap nhat:
[Java, C++, PHP, Pascal]
list sau khi cap nhat:
[Java, C++, PHP, Python]
```




ARRAYLIST – VÍ DỤ

- Đếm số lần xuất hiện của một phần tử trong danh sách

```
import java.util.ArrayList;
public class viDuArrayList_3 {

    public static void main(String args[]) {
        ArrayList<String> list = new ArrayList<String>();
        // thêm các phần tử vào list
        list.add("Java");
        list.add("C++");
        list.add("PHP");
        list.add("Java");
        int count = 0; // khởi tạo biến đếm
        for(int i=0; i < list.size(); i++){
            if(list.get(i).equals("Java")) { // equals hàm kiểm tra 2 chuỗi giống nhau
                count++;
            }
        }
        System.out.println("Số lần xuất hiện của phần tử Java là: "+count);
    }
}
```

```
C:\GiangDay\ViDu_Java>java viDuArrayList_3
Số lần xuất hiện của phần tử Java là: 2
```




ARRAYLIST – VÍ DỤ

- Đếm số lần xuất hiện của một phần tử trong danh sách – Sử dụng Collections

```
import java.util.ArrayList;
import java.util.Collections;
public class viDuArrayList_4 {
```

```
    public static void main(String args[]) {
        ArrayList<String> list = new ArrayList<String>();
        // thêm các phần tử vào list
        list.add("Java");
        list.add("C++");
        list.add("PHP");
        list.add("Java");
        //sử dụng Collections để đếm số lần xuất hiện của 1 phần tử
        int count = Collections.frequency(list, "Java");
        System.out.println("Số lần xuất hiện của phần tử Java là: "+count);
    }
}
```

```
C:\GiangDay\ViDu_Java>java viDuArrayList_4
Số lần xuất hiện của phần tử Java là: 2
```

PHẦN 4. MẢNG CÁC ĐỐI TƯỢNG





KHÁI NIỆM

- Java cho phép lưu trữ các **đối tượng** dưới dạng các phần tử của mảng giống với các kiểu dữ liệu nguyên thủy khác
- **Lưu ý** rằng khi bạn nói ‘mảng đối tượng’, bản thân đối tượng không được lưu trữ trong mảng mà là các tham chiếu của đối tượng.
- Khai báo
 - **Class_name [] objArray;**
 - **Class_name objArray[];**
- Khai báo và khởi tạo:
 - **Class_name [] objArray = new Class_name** [<số phần tử>];
- **Chú ý:** mỗi phần tử của mảng cần được khởi tạo



VÍ DỤ

- Giả sử lớp **Employee** có các thuộc tính, phương thức được khai báo như bên dưới. Viết chương trình khởi tạo một mảng **employee** và in thông tin các đối tượng employee ra màn hình.

```
class Employee{
    int empId;
    String name;
    Employee (int eid, String n){//khởi tạo đối tượng employee
        empId = eid;
        name = n;
    }
    public void showData(){//in thông tin đối tượng employee ra màn hình
        System.out.print ("EmpId = "+empId + " " + " Employee Name = "+name);
        System.out.println();
    }
}
```



VÍ DỤ

```
public class viDuMangDoiTuong {
    public static void main(String args[]) {
        //tạo mảng đối tượng Employee
        Employee[] obj = new Employee[2] ;
        //khởi tạo giá trị cho mỗi phần tử mảng với hàm xây dựng
        obj[0] = new Employee(100,"ABC");
        obj[1] = new Employee(200,"XYZ");

        //in thông tin mỗi phần tử mảng ra màn hình
        System.out.println("Employee Object 1:");
        obj[0].showData();
        System.out.println("Employee Object 2:");
        obj[1].showData();
    }
}

class Employee{
    int empId;
    String name;
    public Employee(int eid, String n){
        empId = eid;
        name = n;
    }
    public void showData(){
        System.out.print("EmpId = "+empId + " " + " Employee Name = "+name);
        System.out.println();
    }
}
```

```
C:\GiangDay\ViDu_Java>java viDuMangDoiTuong
Employee Object 1:
EmpId = 100   Employee Name = ABC
Employee Object 2:
EmpId = 200   Employee Name = XYZ
```

PHẦN 5. SẮP XẾP PHẦN TỬ MẢNG





KHÁI NIỆM

- Là quá trình sắp xếp các phần tử của mảng theo một thứ tự nào đó.
- Quá trình sắp xếp dựa trên các giá trị và quy luật cụ thể.
- Ví dụ:
 - Sắp xếp danh sách các điểm kiểm tra theo **thứ tự tăng dần**.
 - Sắp xếp danh sách sinh viên theo thứ tự bảng chữ cái theo tên của sinh viên.
- Có nhiều thuật toán sắp xếp với độ phức tạp và hiệu quả khác nhau. Tìm hiểu 2 phương pháp
 - **Sắp xếp chọn (Selection Sort)**
 - **Sắp xếp chèn (Insertion Sort)**



SẮP XẾP CHỌN (SELECTION SORT)

- Lấy giá trị đầu danh sách làm giá trị nhỏ nhất (hoặc lớn nhất) tùy yêu cầu sắp xếp.
- Duyệt qua danh sách, khi tìm thấy giá trị nhỏ hơn thì lấy đó làm giá trị nhỏ nhất.
- Lấy phần tử có giá trị nhỏ nhất, đổi vị trí với phần tử đầu danh sách. Lúc này phần tử đầu danh sách đã đúng vị trí.
- Lặp lại quá trình duyệt danh sách, với danh sách bắt đầu từ phần tử thứ 2 trở đi. Kết thúc khi danh sách không còn phần tử.



SẮP XẾP CHỌN (SELECTION SORT)

```
public class viDuSelectionSort {
    static void sort(int arr[])
    {
        int n = arr.length;

        // duyệt từng phần tử mảng chưa được sắp xếp
        for (int i = 0; i < n-1; i++)
        {
            // tìm vị trí phần tử có giá trị nhỏ nhất chưa được sắp đúng vị trí
            int min_idx = i;
            for (int j = i+1; j < n; j++)
                if (arr[j] < arr[min_idx])
                    min_idx = j;

            // đổi vị trí với phần tử đầu của danh sách chưa sắp xếp
            int temp = arr[min_idx];
            arr[min_idx] = arr[i];
            arr[i] = temp;
        }
    }

    // in các phần tử của mảng
    static void printArray(int arr[])
    {
        int n = arr.length;
        for (int i=0; i<n; ++i)
            System.out.print(arr[i]+" ");
        System.out.println();
    }

    public static void main(String args[]) {
        int arr[] = {64,25,12,22,11};
        sort(arr);
        System.out.println("Sorted array");
        printArray(arr);
    }
}
```

```
C:\GiangDay\ViDu_Java>java viDuSelectionSort
Sorted array
11 12 22 25 64
```



SẮP XẾP CHÈN (INSERT SORT)

- Thuật toán sắp xếp bắt chước cách sắp xếp quân bài của những người chơi bài.
- Danh sách cần sắp xếp như bộ bài, và tay chúng ta là một danh sách con.
- **Lần lượt** rút từng quân bài trong bộ bài, **so sánh** với các quân bài đang có trên tay và **sắp xếp** quân bài mới rút vào đúng vị trí trong tập bài trên tay.
- Quá trình sắp xếp **kết thúc** khi **không còn quân bài nào** để rút nữa.

6 5 3 1 8 7 2 4



SẮP XẾP CHÈN (INSERT SORT)

```
public class viDuInsertionSort {
    static void sort(int arr[])
    {
        int n = arr.length;
        for (int i = 1; i < n; ++i) {
            int key = arr[i];
            int j = i - 1;
            /*di chuyển các phần tử đứng trước vị trí i và có giá trị lớn hơn
            phần tử tại vị trí i về sau 1 vị trí.
            cho phép đẩy giá trị key về đầu danh sách*/
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j = j - 1;
            }
            arr[j + 1] = key;
        }
    }
    static void printArray(int arr[])
    {
        int n = arr.length;
        for (int i = 0; i < n; ++i)
            System.out.print(arr[i] + " ");

        System.out.println();
    }
    public static void main(String args[]) {
        int arr[] = { 12, 11, 13, 5, 6 };
        sort(arr);
        printArray(arr);
    }
}
```

```
C:\GiangDay\ViDu_Java>java viDuInsertionSort
5 6 11 12 13
```

Stop!

